Bereich: Übungs-Programmentwurf

SnatChat	**Musterlösung**
Package: `de.dhbwka.java.exams.speedyquiz`	**Klasse:** `SnatChat`

Die Klassen sind nach der Klasse `SnatChat` alphabetisch nach Klassenname geordnet.

```
package de.dhbwka.java.exams.snatchat;

public class SnatChat {

   public static void main( String[] args ) {
      SnatChatRoom room = new SnatChatRoom( "GansGeheim" );

      room.register( new SnatChatWindow( room, new Account( "Bob" ) ) );
      room.register( new SnatChatWindow( room, new Account( "Alice" ) ) );
   }

}
```

```
package de.dhbwka.java.exams.snatchat;

import java.awt.Color;
import java.util.Random;

public class Account {

   private State state = State.AVAILABLE;
   private String name;
   private Color color;

   public Account( String name ) {
      this.name = name;
      Random r = new Random();
      this.color =
            new Color( r.nextInt( 201 ), r.nextInt( 201 ), r.nextInt( 201 ) );
   }

   public String getName() {
      return this.name;
   }

   public Color getColor() {
      return this.color;
   }

   public State getState() {
      return state;
   }

   public void setState( State state ) {
      this.state = state;
   }

}
```

```java
package de.dhbwka.java.exams.snatchat;

public class Message {

   private Account sender;

   private String text;

   public Message( Account sender, String text ) {
      super();
      this.sender = sender;
      this.text = text;
   }

   public String getText() {
      return text;
   }

   public Account getSender() {
      return sender;
   }

   @Override
   public String toString() {
      return this.sender.getName() + ": " + this.text;
   }

   public static String rot13( String text ) {
      char[] textArray = text.toCharArray();

      for ( int i = 0; i < textArray.length; i++ ) {
         char ch = textArray[i];
         if ( ch >= 'a' && ch <= 'm' || ch >= 'A' && ch <= 'M' ) {
            ch += 13;
         } else if ( ch >= 'n' && ch <= 'z' || ch >= 'N' && ch <= 'Z' ) {
            ch -= 13;
         }
         textArray[i] = ch;
      }
      return new String( textArray );
   }

}
```

```java
package de.dhbwka.java.exams.snatchat;

public interface SnatChatFrontend {

   public void receiveMessage( String text );

   public void receiveMessage( Message msg );

   public Account getAccount();
}
```

```java
package de.dhbwka.java.exams.snatchat;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

public class SnatChatRoom {

   private String roomName;

   private List<SnatChatFrontend> frontends = new ArrayList<>();

   public SnatChatRoom( String roomName ) {
      super();
      this.roomName = roomName;
   }

   public String getRoomName() {
      return this.roomName;
   }

   public void sendMessage( Message message ) {
      this.logMessage( message.toString() );
      for ( SnatChatFrontend s : this.frontends ) {
         s.receiveMessage( message );
      }
   }

   public void sendMessage( String text ) {
      this.logMessage( text );
      for ( SnatChatFrontend s : this.frontends ) {
         s.receiveMessage( text );
      }
   }

   public void register( SnatChatFrontend frontend ) {
      this.frontends.add( frontend );

      for ( String logMsg : this.loadLogFile() ) {
         frontend.receiveMessage( logMsg );
      }
   }

   public void unregister( SnatChatFrontend frontend ) {
      this.frontends.remove( frontend );
   }
```

```
   private List<String> loadLogFile() {
      List<String> log = new ArrayList<>();
      try ( BufferedReader br =
            new BufferedReader( new FileReader( this.roomName + ".txt" ) ) ) {
         String line;
         while ( (line = br.readLine()) != null ) {
            log.add( Message.rot13( line ) );
         }
      } catch ( Exception e ) {
      }

      // Erweiterung max 10 Nachrichten
      if ( log.size() > 10 ) {
         log = log.subList( log.size() - 10, log.size() );
      }

      return log;
   }

   private void logMessage( String text ) {
      try ( PrintWriter pw = new PrintWriter(
            new FileWriter( this.roomName + ".txt", true ) ) ) {
         pw.println( Message.rot13( text ) );
      } catch ( Exception e ) {
      }
   }

}
```

```
package de.dhbwka.java.exams.snatchat;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

import javax.swing.ButtonGroup;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JTextField;

import de.dhbwka.java.exams.snatchat.provided.ChatMessagesComponent;

@SuppressWarnings( "serial" )
public class SnatChatWindow extends JFrame implements SnatChatFrontend {

   private SnatChatRoom room;

   private final Account account;
```

```
   private ChatMessagesComponent chatMessagesComponent =
         new ChatMessagesComponent();

   private JTextField messageField = new JTextField();

   private JButton sendButton = new JButton( "Send" );

   public SnatChatWindow( SnatChatRoom room, Account account ) {
      super( String.format( "%s (%s)", account.getName(),
            room.getRoomName() ) );
      this.account = account;
      this.room = room;

      this.initUI();
   }

   private void initUI() {
      this.setLayout( new BorderLayout( 5, 5 ) );

      JLabel labelName = new JLabel( account.getName(), JLabel.CENTER );
      labelName.setForeground( account.getColor() );

      this.add( labelName, BorderLayout.NORTH );
      this.add( this.chatMessagesComponent, BorderLayout.CENTER );

      JPanel panelSouth = new JPanel( new BorderLayout( 5, 5 ) );

      panelSouth.add( this.messageField, BorderLayout.CENTER );
      panelSouth.add( this.sendButton, BorderLayout.EAST );

      ButtonGroup btnGrp = new ButtonGroup();
      JPanel panStateRadios = new JPanel( new FlowLayout() );
      for ( final State s : State.values() ) {
         JRadioButton radioButton = new JRadioButton( s.getLabel() );
         btnGrp.add( radioButton );
         radioButton.setSelected( s == account.getState() );
         radioButton.addActionListener( new ActionListener() {
            @Override
            public void actionPerformed( ActionEvent e ) {
               account.setState( s );
               room.sendMessage(
                     String.format( "State of user '%s' is now '%s'",
                           account.getName(), s.getLabel() ) );
            }
         } );
         panStateRadios.add( radioButton );
      }
      panelSouth.add( panStateRadios, BorderLayout.NORTH );

      this.add( panelSouth, BorderLayout.SOUTH );

      this.sendButton.addActionListener( new ActionListener() {
         @Override
         public void actionPerformed( ActionEvent e ) {
            sendMessage();
         }
      } );
```

```
    // Zusatzpunkt
    this.messageField.addKeyListener( new KeyAdapter() {
       @Override
       public void keyPressed( KeyEvent e ) {
          if ( e.getKeyCode() == KeyEvent.VK_ENTER ) {
             sendMessage();
          }
       }
    } );

    this.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
    this.setSize( 325, 300 );
    this.setVisible( true );
 }

 @Override
 public void receiveMessage( String text ) {
    this.receiveMessage( text, Color.GRAY );
 }

 @Override
 public void receiveMessage( Message msg ) {
    this.receiveMessage( msg.toString(), msg.getSender().getColor() );
 }

 @Override
 public Account getAccount() {
    return this.account;
 }

 private void receiveMessage( final String text, Color color) {
    JLabel messageLabel = new JLabel( text );
    messageLabel.setForeground( color );

    new Thread() {
       public void run() {

          int seconds = 30;
          while ( seconds >= 0 ) {
             messageLabel.setText( text + " [ " + seconds-- + " ]" );
             try {
                Thread.sleep( 1000 );
             } catch ( InterruptedException e ) {
             }
          }
          SnatChatWindow.this.chatMessagesComponent.remove( messageLabel );
       };
    }.start();

    this.chatMessagesComponent.add( messageLabel );
 }
```

```
   private void sendMessage() {
      if ( !this.messageField.getText().isEmpty() ) {
         this.room.sendMessage(
               new Message( this.account, this.messageField.getText() ) );
         this.messageField.setText( "" );
      } else {
         JOptionPane.showMessageDialog( this,
               String.format( "Dear %s, please enter a message",
                     this.account.getName() ),
               "Warning!", JOptionPane.WARNING_MESSAGE );
      }
   }

}
```

```
package de.dhbwka.java.exams.snatchat;

public enum State {

   AVAILABLE("Available"), AWAY("Away"), DND("Do not disturb");

   private final String label;

   private State( String label ) {
      this.label = label;
   }

   public String getLabel() {
      return label;
   }

}


// Bereitgestellte Klasse folgt noch
```

Bereitgestellte UI-Klasse für Chat-Nachrichten

```java
package de.dhbwka.java.exams.snatchat.provided;

import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;

import javax.swing.BorderFactory;
import javax.swing.JComponent;
import javax.swing.JPanel;
import javax.swing.JScrollPane;

/**
 * Bereitgestelltes Oberflaechenelement zur Anzeige der Chat-Nachrichten auf
 * Basis einer beliebigen Komponente.
 *
 * Einfach wie ein Panel benutzen:
 *
 * <pre>
 * ChatMessagesComponent chats = new ChatMessagesComponent();
 * chats.add(...);
 * (...)
 * chats.remove(...);
 * </pre>
 */
@SuppressWarnings( "serial" )
public class ChatMessagesComponent extends JScrollPane {

   /**
    * Anzahl der Nachtrichten
    */
   private int messagesCount = 0;

   /**
    * Panel, zu dem die Komponenten hinzugefuegt werden
    */
   private JPanel messagesPanel = new JPanel( new GridBagLayout() );

   /**
    * Auffuellkomponente in Y-Richtung (damit Nachrichten oben stehen)
    */
   private JPanel fillerY = new JPanel();

   /**
    * Erstelle die Komponente fuer die Chat-Nachrichten
    */
   public ChatMessagesComponent() {
      this.setViewportView( this.messagesPanel );
   }
```

```
    /**
     * Fuege eine Komponente als Chat-Nachricht hinzu
     *
     * @param component
     *           Komponente zum hinzufuegen
     */
    public void add( JComponent component ) {
        this.messagesPanel.remove( this.fillerY );
        component.setBorder( BorderFactory.createEmptyBorder( 1, 0, 1, 0 ) );
        this.messagesPanel.add( component,
                new GridBagConstraints( 0, this.messagesCount++, 1, 1, 1., 0.,
                        GridBagConstraints.BASELINE_LEADING,
                        GridBagConstraints.HORIZONTAL, new Insets( 2, 2, 2, 2 ), 0,
                        0 ) );
        this.messagesPanel.add( this.fillerY,
                new GridBagConstraints( 0, this.messagesCount, 1, 1, 1., 1.,
                        GridBagConstraints.BASELINE_LEADING, GridBagConstraints.BOTH,
                        new Insets( 2, 2, 2, 2 ), 0, 0 ) );
        this.messagesPanel.revalidate();
    }

    /**
     * Entferne eine Komponente
     *
     * @param component
     *           Komponente die entfernt werden soll
     */
    public void remove( JComponent component ) {
        this.messagesPanel.remove( component );
        this.messagesPanel.revalidate();
        this.messagesPanel.repaint();
    }

}
```